



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A MINOR PROJECT MID-TERM PROGRESS REPORT
ON
MULTI-AGENT DATA ANALYSIS AND INSIGHT GENERATION PLATFORM FOR
FINANCIAL AND SALES DATA**

Submitted By:

Prashant Kafle (THA080BCT026)

Roshan Poudel (THA080BCT036)

Santosh Khadka (THA080BCT039)

Sushil Bhatta (THA080BCT048)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

July 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Thapathali Campus, Institute of Engineering, Tribhuvan University, for providing us the opportunity to work on this minor project.

We are deeply indebted to our supervisor, Er. Rajad Shakya, for his guidance, constructive feedback, and continuous encouragement throughout the development of this project. His suggestions have helped us refine both the technical design and the report structure.

We also thank the faculty members of the department for creating an academic environment that supports practical engineering work. Finally, we are grateful to our friends and families for their patience and support during this phase of the project.

Submitted By:

Prashant Kafle (THA080BCT026)

Roshan Poudel (THA080BCT036)

Santosh Khadka (THA080BCT039)

Sushil Bhatta (THA080BCT048)

July 2026

ABSTRACT

This project develops a multi-agent data analysis pipeline that turns raw CSV-based financial and sales data into structured insights and charts. The system is designed around a shared state model and a planned sequence of six specialized agents: structural profiling, semantic tagging, preprocessing, statistical analysis, validation, and report generation. Each agent performs one focused responsibility and passes its results to the next stage through a controlled pipeline.

At the mid-point of development, the first four agents are implemented and the backend pipeline runs end to end from ingestion through statistical analysis. The codebase now includes robust CSV and Excel loading, semantic metadata generation, deterministic preprocessing, and statistical analysis with chart generation. Output validation (Agent 5) and report generation (Agent 6) are not yet implemented and form the main remaining work, together with refinement, testing, and evaluation on more datasets. The project is moving toward a practical analytics tool that can reduce manual effort and make technical analysis more accessible to non-technical users.

Keywords: Multi-agent systems, data analysis, financial reporting, sales analytics, LangGraph, preprocessing, validation, automated reporting

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Definition	1
1.4 Objectives	2
1.5 Scope and Applications	2
2 LITERATURE REVIEW	4
2.1 Traditional Data Analytics Platforms	4
2.2 AI-Assisted Analytics and Large Language Model-Based Data Analysis	4
2.3 Automated Data Visualization from Tabular/CSV Data	5
2.4 Agentic AI Frameworks and Multi-Agent Systems	5
2.5 Existing Research on Automated Data Analysis Systems	6
2.6 Research Gap and Project Positioning	6
3 REQUIREMENT ANALYSIS	7
3.1 Functional Requirements	7
3.2 Non-Functional Requirements	7
3.3 Requirement Status	8
4 THEORETICAL APPROACH	9
4.1 Large Language Model Theory	9
4.1.1 From Transformers to Modern LLMs	9
4.1.2 Decoder-Only Transformer Architecture	9

4.1.3	Tokenization and Vocabulary	10
4.1.4	Embedding Layer	10
4.1.5	Positional Encoding	10
4.1.6	Self-Attention and Multi-Head Attention	10
4.1.7	Feed-Forward Network	11
4.1.8	Residual Connections and Layer Normalization	11
4.1.9	Logits, Softmax, and Next-Token Prediction	12
4.2	Ollama Model and Inference Architecture	13
4.2.1	Why Ollama Is Used	14
4.2.2	Local Model Loading and REST API	14
4.2.3	Prompt Processing Pipeline	14
5	SYSTEM ARCHITECTURE AND METHODOLOGY	16
5.1	Overall Pipeline	16
5.2	Shared State Design	18
5.3	Methodology	18
6	IMPLEMENTATION DETAILS	19
6.1	Agent 1: Structural Profiler	19
6.2	Agent 2: Semantic Tagger	20
6.3	Agent 3: Preprocessor	21
6.4	Agent 4: Statistical Analysis	22
6.5	Current Implementation Snapshot	23
7	RESULTS AND ANALYSIS	24
7.1	Observed Progress	24
7.2	Practical Output	24
7.3	Analysis	24
7.4	Current Challenges	25
8	REMAINING TASKS	26
9	CONCLUSION	27
	APPENDICES	28
A	COST ESTIMATE	28
B	PROJECT TIMELINE	29

LIST OF FIGURES

4.1	Decoder-only Transformer architecture used by modern LLMs	13
4.2	Local Transformer inference pipeline executed by Ollama	15
5.1	System architecture and agent flow	16
5.2	Agent workflow	17
6.1	Workflow of Agent 1: Structural Profiling	19
6.2	Workflow of Agent 2: Semantic Tagging	20
6.3	Workflow of Agent 3: Preprocessing Pipeline	21
6.4	Workflow of Agent 4: Statistical Analysis	22

LIST OF TABLES

3.1	Current requirement coverage in the codebase	8
6.1	Implementation snapshot at the mid-point	23
A.1	Indicative project cost estimate	28
B.1	Simplified project timeline	29

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BI	Business Intelligence
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DOECE	Department of Electronics and Computer Engineering
HTML	HyperText Markup Language
IOE	Institute of Engineering
JSON	JavaScript Object Notation
LLM	Large Language Model
PDF	Portable Document Format
TU	Tribhuvan University

1. INTRODUCTION

1.1. Background

Data-driven decision-making has become a foundational requirement for organizational effectiveness in health, agriculture, education, and commerce. In Nepal, the government's Digital Nepal Framework explicitly prioritizes data-centric governance across these sectors. However, the operational reality for most organizations particularly those outside the Kathmandu Valley remains characterized by manual spreadsheet management and underutilized datasets. The technical barrier to entry for modern analytics is high: tools demand proficiency in query languages, schema design, or statistical software, while enterprise platforms carry licensing costs that are unsustainable for donor-funded non-governmental organizations (NGOs) and municipal bodies. The emergence of LLMs and agentic orchestration frameworks has created a new design space for autonomous software systems [1]. LLMs can now interpret column semantics and narrate statistical findings in natural language, while agentic frameworks such as LangGraph enable reliable, stateful, multi-step automation. The convergence of these technologies makes it feasible to build a pipeline that replicates the workflow of a human data analyst validation, cleaning, profiling, visualization, statistical testing, interpretation, and reporting without requiring human intervention at each step.

1.2. Motivation

The motivation for this project is based on the following points:

- Despite the availability of data, many organizations lack the technical expertise required to analyze and interpret it effectively.
- Traditional data analysis tools often require statistical knowledge, programming skills, or manual report interpretation.
- Existing AI-based systems using LLMs may generate unverified or hallucinated insights, leading to unreliable conclusions.
- Recent advances in Artificial Intelligence and Multi-Agent Systems provide an opportunity to automate complex analytical tasks.

1.3. Problem Definition

Organizations accumulate data daily sales records, transaction logs, expense sheets, customer information yet cannot efficiently convert this data into insights that drive strategy and decision-making.

Existing challenges:

1. **Manual workflows:** Data analysis currently requires skilled staff to manually clean data, calculate metrics, build charts, and compile reports a process taking days to weeks
2. **Lack of standardization:** Without a systematic methodology, different people produce inconsistent results, making it difficult to compare periods or validate findings
3. **Limited accessibility:** Analytics tools require specialized knowledge. Non-technical stakeholders cannot ask ad-hoc questions without analyst intermediaries

This project proposes an end-to-end automated system that takes raw sales and financial CSV data and produces validated, auditable, professionally formatted reports with trend insights and recommendations.

1.4. Objectives

The primary objectives of this project are:

1. Develop an autonomous multi-agent system that converts raw sales and financial data (CSV format) into professional analytical reports and forecasts without manual coding
2. Democratize basic financial analytics and sales optimization for small to medium businesses and organizations through a no-code web interface

1.5. Scope and Applications

Technical Scope:

- Supports common business data types: sales records, transaction logs, expense sheets, in CSV format
- A six-agent pipeline design; four agents (profiling, semantic tagging, preprocessing, and statistical analysis) are currently implemented, while validation and report generation are planned
- Runs as web application without requiring to download additional software

Primary Use Cases:

1. **Sales Performance Tracking:** Automated analysis of sales data to identify trends, top-performing products, seasonal patterns, and revenue forecasts for planning

2. **Financial Reporting & Budgeting:** Generation of expense summaries, income vs. expenditure analysis, budget tracking, and monthly/quarterly financial reports for small to medium organizations
3. **Retail & E-commerce Analytics:** Customer purchase behavior, inventory turnover analysis, demand forecasting, and sales channel comparison
4. **Small Business Operations:** Cash flow trends, profit margin analysis, supplier/payment tracking, and operational dashboards without requiring accounting software expertise
5. **Financial Data for Non-Finance Teams:** Simplified reports that translate raw financial/sales data into plain-language insights for managers, project leads, and decision-makers

2. LITERATURE REVIEW

2.1. Traditional Data Analytics Platforms

Platforms for Business Intelligence like Microsoft Power BI, Tableau, and Google Looker Studio are popular choices when it comes to data visualization, creation of dashboards, and decision making. These technologies offer a variety of data connectivity options, visualization functions, and reporting features, helping companies track their key performance indicators and business processes. With all their ecosystem and integration possibilities, these BI systems can be used by large corporations. However, there is a number of disadvantages of BI systems as well. First of all, users need to know database schema, configure their dashboards, and prepare the data themselves. Moreover, licensing and other infrastructure expenses may prove to be expensive for small companies, educational institutions, and non-profit organizations lacking funds. However, one of the major drawbacks of traditional BI systems is their lack of automation in interpretation, relying only on visualization. According to Dwivedi et al. [1], large language models are changing data analytics. LLMs let us interact naturally, reason automatically, and get smart decision support. The authors argue that we need to progress from static dashboards to AI systems that offer insightful auto-recommendations. This points out an important research gap. To address it, we should develop intelligent, multi-agent data analyst systems.

2.2. AI-Assisted Analytics and Large Language Model-Based Data Analysis

LLMs are changing data analytics with their ability to chat naturally about data [1]. Tools like ChatGPT with Code Interpreter and other assistant-style systems let people upload datasets and get back code, charts, stats analyses, and plain text explanations [2]. This is big because it makes data analysis easier for those without a ton of technical skills.

One huge plus is how easy these platforms are to use. Regular folks can talk to a dataset without needing loads of coding knowledge. Researchers show that LLMs can handle lots of jobs, like cleaning data, doing exploratory analysis, spitting out visuals, and even putting together reports [3].

Still, there are some downsides. For one, getting the same result each time is tricky since outcomes rely heavily on the questions you ask. You might phrase things differently and end up with entirely distinct findings. Also, these systems tend to be single agent setups that don't follow a strict workflow or include checks for accuracy. There's no built-in QA process, and sometimes the code generated can have errors or missing steps [4, 5]. This really drives the need for more complex systems that employ different agents

for doing analysis, verifying results, and generating reports.

2.3. Automated Data Visualization from Tabular/CSV Data

Several studies have investigated automated visualization generation from structured datasets. Data2Vis [6] proposed a sequence-to-sequence neural architecture that translates datasets into Vega-Lite visualization specifications. VizML [7] applied machine learning to learn visualization design choices from over one million dataset-visualization pairs. Draco [8] formalized visualization design knowledge as logical constraints to recommend effective visual encodings. DeepEye [9] further automated chart generation and ranking. These studies collectively demonstrate the feasibility of automated visualization generation and provide strong theoretical support for the Visualization Agent proposed in this research.

The main insight from this line of work is that visualization creation can be treated as a recommendation or generation task rather than a purely manual design task. This is especially relevant for CSV-based analysis workflows, where the system must decide whether to use bar charts, line charts, scatter plots, or other encodings based on the structure and semantics of the tabular data. The proposed project adopts this idea by using an agent that converts cleaned tabular outputs into appropriate static visualizations for reports.

2.4. Agentic AI Frameworks and Multi-Agent Systems

The newest stuff in Agentic AI includes LangGraph, CrewAI, and AutoGen frameworks that help different AI agents work together on big projects [10, 11]. Traditionally, one AI does all the work, but these new setups let multiple agents take on tasks based on what they're best at.

LangGraph helps manage agent teamwork with things like conditional routing and checkpointing. Meanwhile, AutoGen is all about letting agents chat back and forth, and CrewAI handles task delegation based on roles. Altogether, these tools show promise for building really smart, self-reliant systems that can figure out tough problems, plan, and coordinate tasks efficiently.

According to Wu et al. [11], when you set up proper structures for AI conversations, teams of specialized agents can ace complex assignments. This covers everything from coding and math to answering questions and making decisions. It's clear that such collaborations do better than lone agents.

Still, most work right now only applies these frameworks to areas like software engineering, web searches, Q&A, and general reasoning. Not many have looked into how well they fit into fixed data analysis processes that mix deterministic stats ops with more uncertain language-model thinking. This gap presents an opportunity for the

development of multi-agent systems specifically designed for automated data analysis and business intelligence workflows.

2.5. Existing Research on Automated Data Analysis Systems

Researchers are looking into artificial intelligence to make data analytics easier. They're mixing machine learning, big language models, and natural language processing to help with exploring data, making it visual, and getting insights.

Sun et al. came up with LAMBDA, a Large Model Based Data Agent, which uses special AI agents to do data analysis [3]. These agents follow instructions in regular language to do tasks like code generation, stats, and visualizations. The cool thing is, when these agents work together, they improve accuracy and dependability more than solo agents [3].

While there's been progress, most systems now only cover certain parts of the analytics job. For instance, statistical tools are great but aren't very user-friendly, and some language model tools don't always deliver consistent results because they rely too much on how you write prompts. Also, many current tools work in isolation they don't all come together to form that full analytics process which would include everything from pre-processing data to generating reports.

So, we need a better way. There's a call for a complete multi-agent setup that provides reliable and repeatable data analytics. This could revolutionize how we handle big data tasks.

2.6. Research Gap and Project Positioning

According to the literature reviewed, there is a lack of studies dedicated to multi-agent data processing systems that combine deterministic analytical algorithms with LLM-based reasoning in a reliable end-to-end analytics pipeline [3, 11]. Traditional BI tools require significant user involvement and technical expertise [1]. While LLM-based systems improve accessibility, they do not inherently guarantee reproducibility, auditability, and structured execution workflows [4, 5]. Multi-agent frameworks provide orchestration capabilities but have primarily been applied to software engineering and general reasoning tasks rather than autonomous data analytics and reporting [11, 10]. Therefore, this project proposes a LangGraph-based multi-agent analytics system in which specialized agents perform data ingestion, preprocessing, analysis, visualization, validation, and report generation. Deterministic analytical methods will be used for statistical computations, while an LLM-based agent will generate insights and explanations. The proposed system aims to improve reproducibility, accessibility, and automation of data analytics workflows for educational institutions, small businesses, and non-technical users.

3. REQUIREMENT ANALYSIS

3.1. Functional Requirements

- Upload or load a structured dataset.
- Profile the dataset structure and missingness.
- Infer semantic metadata for columns.
- Preprocess and clean the data.
- Generate statistical summaries and visualizations.
- Validate the outputs before final reporting.
- Produce a final executive report.

3.2. Non-Functional Requirements

- Reproducible execution through shared state and deterministic processing.
- Extensible agent-based design.
- Graceful handling of missing data, invalid input, and validation failures.
- Clear audit trail for preprocessing and validation decisions.

The most important non-functional requirement is traceability. The system does not hide intermediate decisions inside a black box; instead, it keeps outputs in the shared state and logs important actions as the pipeline proceeds. A second important requirement is robustness, because a report-generating system is only useful if it can continue operating sensibly when some inputs are incomplete or imperfect.

3.3. Requirement Status

Table 3.1: Current requirement coverage in the codebase

Requirement	Status	Notes
Data loading and profiling	Implemented	Supports CSV and Excel inputs
Semantic tagging	Implemented	Heuristics plus LLM fallback
Preprocessing and cleaning	Implemented	Audit trail and quality scoring included
Statistics and charts	Implemented	Multiple chart and analysis types available
Validation gate	In progress	Agent 5 not yet implemented
Final report generation	In progress	Agent 6 not yet implemented
Frontend dashboard	Planned	Not yet the main focus
Multi-format expansion	Planned	Future support target

4. THEORETICAL APPROACH

The implementation in this project depends on two theoretical foundations: the internal mechanics of Transformer-based Large Language Models (LLMs), which power the semantic tagging stage, and the local inference pipeline provided by Ollama, which is treated as the official LLM backend of the system. This chapter explains only the concepts that are directly required to understand how the project generates schema metadata and grounded narrative text, keeping the treatment implementation-oriented rather than exhaustive.

4.1. Large Language Model Theory

A Large Language Model is a neural network trained to model the probability distribution of natural language, so that given a sequence of previous tokens it can estimate the most likely next token. In this project the LLM is not used to write free-form prose only; it is used as a structured reasoning component that reads column metadata and returns a strict JSON schema blueprint. Understanding how the model turns text into predictions is therefore necessary to explain why the semantic tagging agent is reliable enough to drive deterministic preprocessing.

4.1.1. *From Transformers to Modern LLMs*

Early sequence models such as recurrent neural networks (RNNs) and long short-term memory (LSTM) networks processed tokens one step at a time. This sequential dependency made training slow and made it difficult to capture relationships between tokens that are far apart in a sentence. The Transformer architecture removed recurrence entirely and replaced it with a self-attention mechanism that relates every token to every other token in parallel. Because all positions are processed simultaneously, Transformers train efficiently on modern hardware and model long-range dependencies directly. Modern LLMs such as the Llama family are scaled Transformer networks trained on large text corpora, and they are the class of models executed locally through Ollama in this project.

4.1.2. *Decoder-Only Transformer Architecture*

The models used for text generation, including Llama, adopt a *decoder-only* Transformer design. The network is a stack of identical decoder blocks, each containing a masked multi-head self-attention sub-layer and a position-wise feed-forward sub-layer. A causal mask ensures that the prediction for a token at position t can only attend to tokens at positions 1 to t , never to future tokens. This left-to-right constraint is what makes the model autoregressive and suitable for generating one token at a time.

4.1.3. Tokenization and Vocabulary

Before any computation, raw text is split into discrete units called tokens using a subword tokenizer (typically byte-pair encoding). Each unique token maps to an integer index in a fixed *vocabulary* of size V . Subword tokenization keeps the vocabulary compact while still being able to represent rare or unseen words as combinations of smaller units. In the project, the prompt sent to the model, which contains column names, inferred types, and sample values, is tokenized in exactly this way before inference.

4.1.4. Embedding Layer

Each token index is converted into a dense vector using a learned embedding matrix $E \in \mathbb{R}^{V \times d}$, where d is the model dimension:

$$x = E(\text{token}) \quad (4.1)$$

Here token is the integer index of the token, E is the embedding lookup table, and $x \in \mathbb{R}^d$ is the resulting embedding vector. This vector is the numerical representation of the token that the rest of the network operates on.

4.1.5. Positional Encoding

Because self-attention is order-agnostic, positional information must be injected explicitly. Sinusoidal positional encodings add a position-dependent signal to each embedding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (4.2)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (4.3)$$

where pos is the position of the token in the sequence, i is the dimension index, and d is the model dimension. Even dimensions use a sine function and odd dimensions use a cosine function. The encoding is added to the token embedding so that the model can distinguish tokens by their position as well as their identity.

4.1.6. Self-Attention and Multi-Head Attention

Self-attention lets each token gather information from other tokens. From the input, three projections are formed: queries Q , keys K , and values V . The scaled dot-product attention is defined as:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (4.4)$$

Here Q , K , and V are matrices of query, key, and value vectors, d_k is the dimension of the key vectors, and $\sqrt{d_k}$ is a scaling factor that prevents the dot products from becoming too large and destabilising the softmax. The term $QK^\top$ measures the similarity between each query and every key; the softmax converts these similarities into weights that are used to form a weighted sum of the value vectors.

Rather than performing attention once, the Transformer uses several parallel attention *heads*, each with its own projections, and concatenates their outputs:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (4.5)$$

where each $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, h is the number of heads, W_i^Q, W_i^K, W_i^V are per-head projection matrices, and W^O is the output projection. Multiple heads allow the model to attend to different types of relationships simultaneously.

4.1.7. Feed-Forward Network

Each decoder block applies a position-wise feed-forward network (FFN) independently to every token vector:

$$\text{FFN}(x) = W_2(\text{ReLU}(W_1x + b_1)) + b_2 \quad (4.6)$$

where x is the token representation, W_1 and W_2 are weight matrices, b_1 and b_2 are bias vectors, and ReLU is the rectified linear activation. The FFN expands the representation to a higher dimension, applies a non-linearity, and projects it back, giving the model additional capacity to transform each token.

4.1.8. Residual Connections and Layer Normalization

To train deep stacks stably, each sub-layer output is added back to its input through a residual connection and then normalized:

$$\text{LayerNorm}(x + \text{Attention}(x)) \quad (4.7)$$

The residual term x preserves the original signal and helps gradients flow through many layers, while layer normalization rescales the summed vector to have a stable mean and variance:

$$\text{LayerNorm}(u) = \gamma \frac{u - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (4.8)$$

where u is the input vector, μ and σ^2 are its mean and variance, ϵ is a small constant for numerical stability, and γ and β are learned scale and shift parameters.

4.1.9. Logits, Softmax, and Next-Token Prediction

After the final decoder block, a linear projection maps the last token representation to a vector of *logits* z of length V , one score per vocabulary token. A softmax converts these logits into a probability distribution:

$$P(y_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad (4.9)$$

where z_i is the logit for token i and the denominator sums over all vocabulary tokens j . The model then predicts the next token conditioned on all previous tokens:

$$P(x_t | x_{<t}) \quad (4.10)$$

where x_t is the token being predicted and $x_{<t}$ denotes all tokens before position t . Text is generated *autoregressively*: the predicted token is appended to the sequence and fed back in to predict the following token, repeating until a stopping condition is reached.

During the model’s original training, these predictions are optimised with the cross-entropy loss:

$$L = - \sum_i y_i \log(p_i) \quad (4.11)$$

where y_i is 1 for the correct next token and 0 otherwise, and p_i is the predicted probability for token i . This objective drives the probability mass toward the correct token. The project itself performs inference only and does not train the model, but the loss explains why the pre-trained model assigns high probability to coherent, well-formed output such as the valid JSON schema the tagging agent depends on.

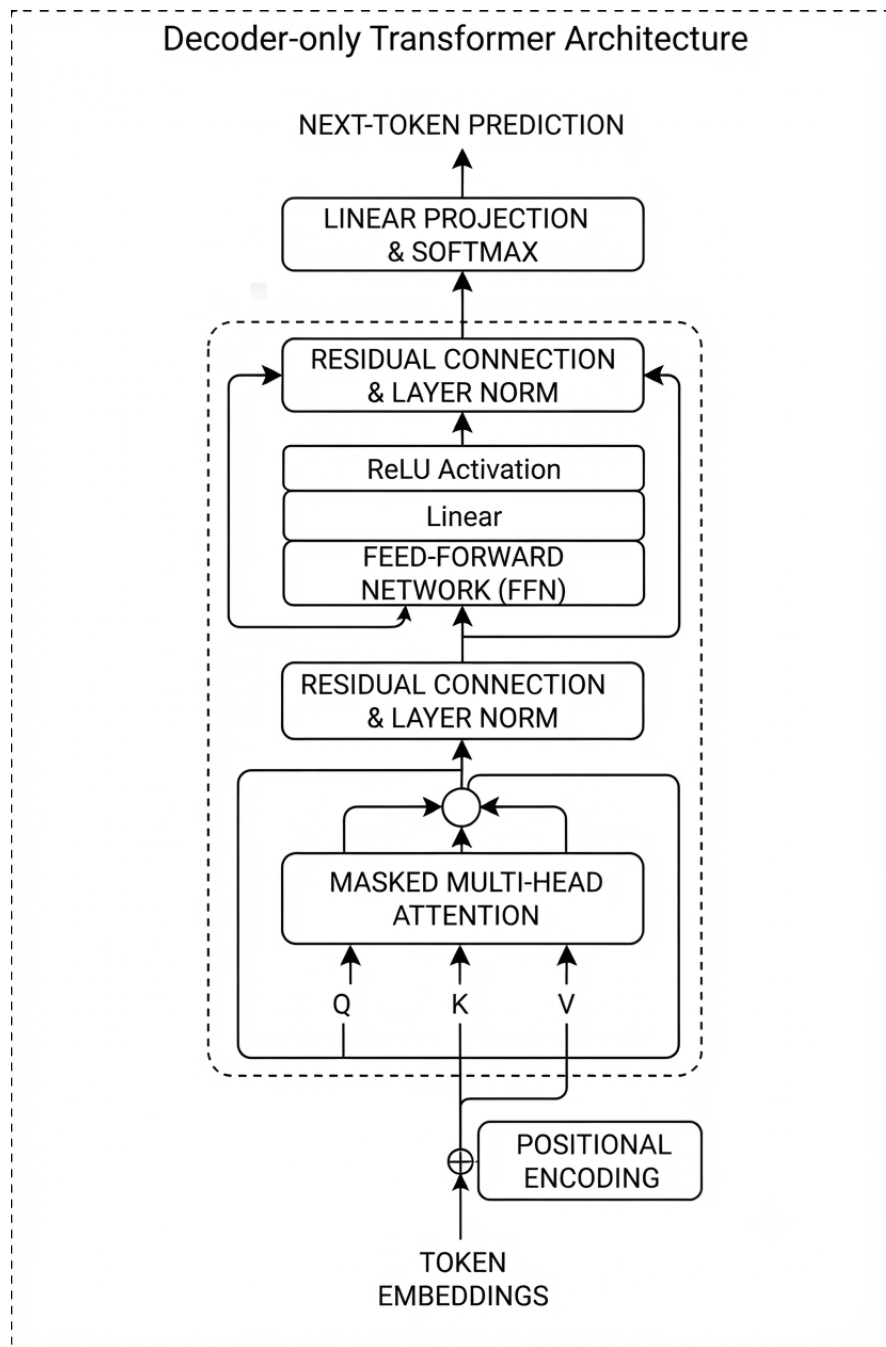


Figure 4.1: Decoder-only Transformer architecture used by modern LLMs

4.2. Ollama Model and Inference Architecture

Ollama is the official LLM backend of this system. It is a lightweight runtime that loads and serves open-source Transformer models, such as the Llama family, entirely on the local machine and exposes them through a local REST API. Rather than viewing Ollama merely as a piece of software, this section describes the full Transformer inference pipeline that Ollama executes when the semantic tagging agent sends it a prompt.

4.2.1. Why Ollama Is Used

The project uses Ollama because it performs inference locally, which gives the system four practical benefits. First, *privacy*: the dataset-derived prompts never leave the machine, which matters for financial and sales data. Second, *offline execution*: the pipeline can run without internet access once the model is pulled. Third, *low and predictable latency and cost*: there are no per-request API charges or network round-trips to a cloud provider. Fourth, *easy deployment*: a single runtime manages model loading and serving, which simplifies reproducing the system. These properties make Ollama an appropriate production backend for an analytics tool intended for small organizations.

4.2.2. Local Model Loading and REST API

When a model such as Llama 3.x is requested, Ollama loads the model weights from local storage into CPU or GPU memory and keeps them resident so repeated requests avoid reload overhead. Inference can run on CPU or be accelerated on a GPU, and memory usage is dominated by the model weights plus the attention cache described below. Agent 2 communicates with the running model through Ollama’s local REST API: it issues a chat-completion request containing a system instruction and a user message, and receives the generated text in response. From the application’s perspective this is a simple local HTTP call, while all the Transformer computation happens inside the Ollama process.

4.2.3. Prompt Processing Pipeline

Inference proceeds through the same Transformer stages described in the previous section, executed in the forward direction only.

Tokenization. The incoming prompt is split into subword tokens and mapped to integer indices in the model’s vocabulary.

Embedding generation. Each token index is converted into a vector through the embedding matrix, $x = E(\text{token})$, and positional information is added using the sinusoidal encoding $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$ and $PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$.

Decoder stack and multi-head attention. The embedded sequence passes through the stack of decoder blocks. In each block, masked multi-head self-attention computes $\text{Attention}(Q, K, V) = \text{Softmax}(QK^\top / \sqrt{d_k})V$, letting each token attend to all earlier tokens in the prompt.

Feed-forward network. Each block then applies $\text{FFN}(x) = W_2(\text{ReLU}(W_1x + b_1)) + b_2$ to every token position.

Residual connections and layer normalization. Every sub-layer is wrapped with a residual connection and normalization, $\text{LayerNorm}(x + \text{Attention}(x))$, keeping the deep

computation numerically stable during inference.

Logits, softmax, and token sampling. After the final block, the last position is projected to logits over the vocabulary and normalized with the softmax $P(y_i) = \exp(z_i) / \sum_j \exp(z_j)$, producing the next-token distribution $P(x_t | x_{<t})$. A sampling strategy (for example temperature-controlled sampling) selects the next token from this distribution. The project sets a low temperature so that the output is nearly deterministic, which is important because the response must be strictly valid JSON.

KV cache. To generate efficiently, Ollama stores the key and value tensors computed for previous tokens in a *KV cache*. When the next token is produced, only the new token's query must attend to the cached keys and values, so the model does not recompute attention over the entire sequence at every step. This is the main optimization that makes autoregressive local generation fast.

Autoregressive generation and response decoding. The sampled token is appended to the sequence and fed back into the model to predict the following token. This loop continues until an end-of-sequence token or length limit is reached. The generated token indices are finally decoded back into text and returned over the REST API to Agent 2, which parses the JSON schema blueprint from it.

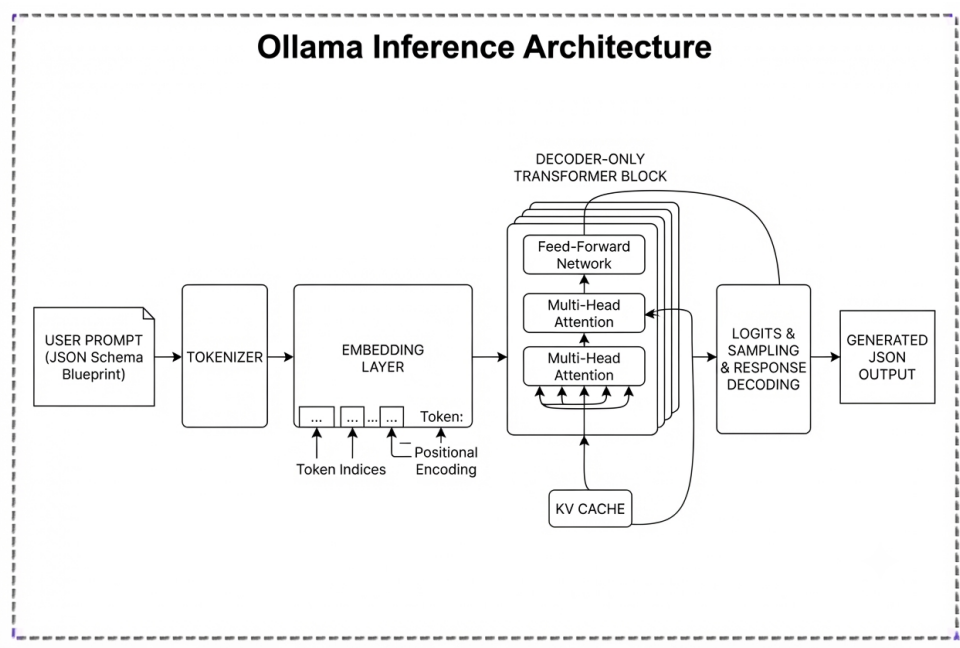


Figure 4.2: Local Transformer inference pipeline executed by Ollama

5. SYSTEM ARCHITECTURE AND METHODOLOGY

5.1. Overall Pipeline

The backend pipeline is built as a directed sequence of agents. Each agent consumes the current GraphState, writes its own outputs back into the shared state, and passes the result to the next step. This keeps the workflow clear and makes debugging easier.

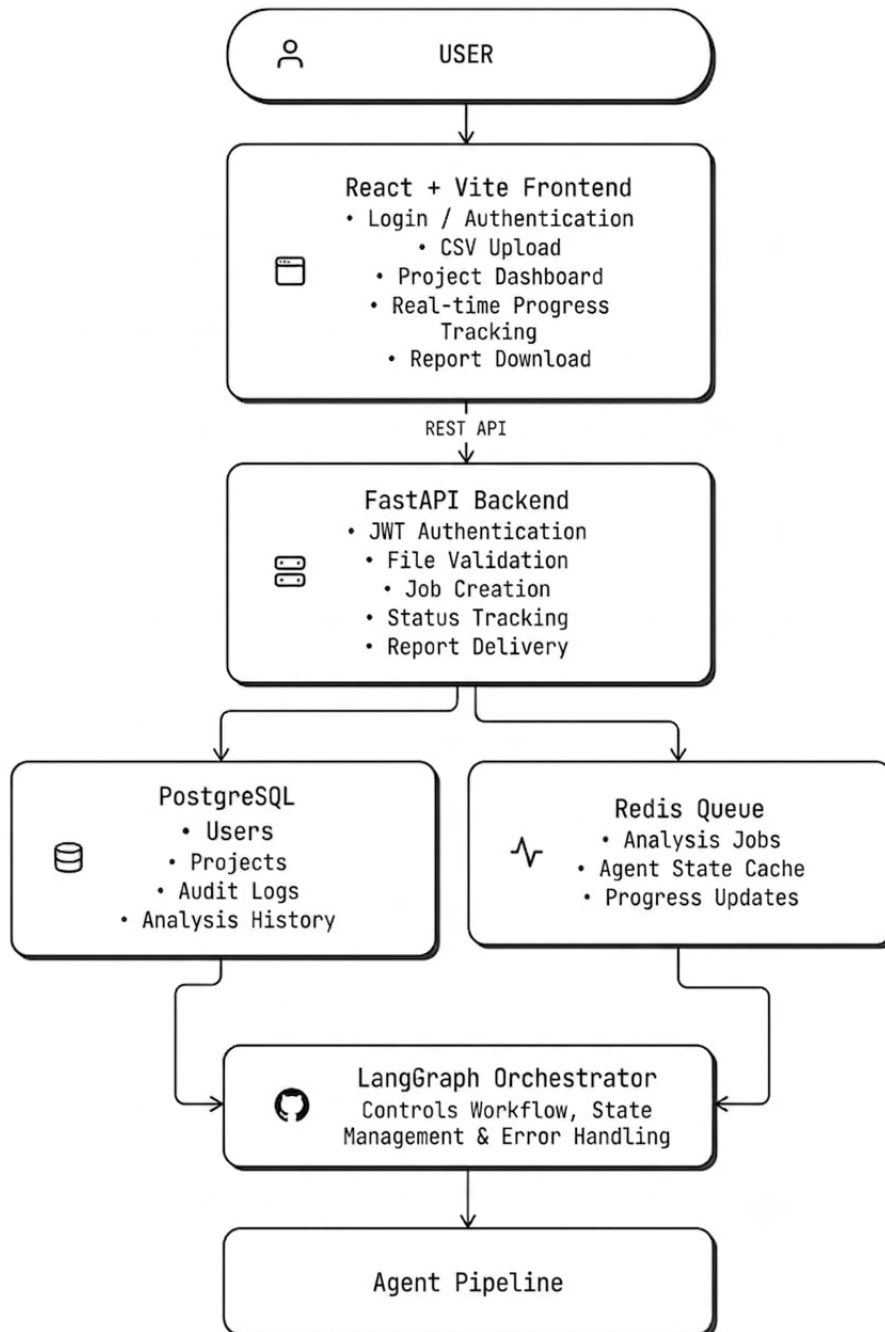


Figure 5.1: System architecture and agent flow

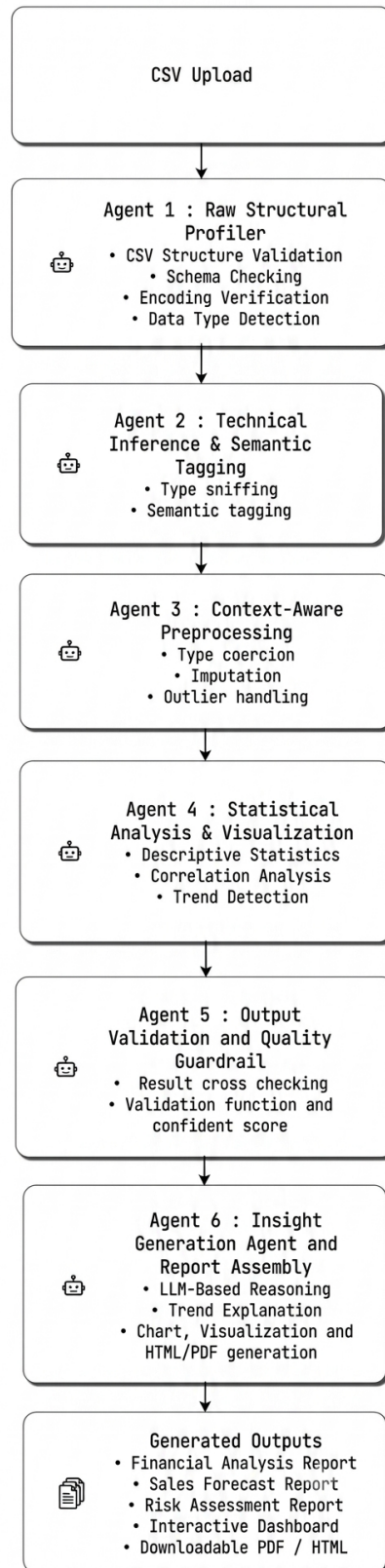


Figure 5.2: Agent workflow

5.2. Shared State Design

The shared state contains the input path, the raw profile, the semantic blueprint, the cleaned dataset, the scaling parameters, the preprocessing log, the quality score, the statistics, the chart paths, an error list, and reliability metadata. This design allows each agent to focus on one responsibility while still keeping the whole workflow connected. Fields for the validation result and the final report path will be added to the state once Agent 5 and Agent 6 are implemented.

The state also carries reliability metadata. Each stage can contribute a confidence score and evidence list, which helps the system decide whether a later stage should proceed. That is useful in the report because it shows that the project is not just generating outputs, but also measuring the reliability of those outputs.

5.3. Methodology

The methodology combines deterministic data processing with LLM-assisted semantic reasoning. Structural checks, preprocessing rules, and statistics are implemented in Python, while the validation and reporting stages remain planned work. Semantic interpretation uses a local LLM served through Ollama, and the planned final narrative generation will use the same backend; both stages are backed by fallback logic so the pipeline remains usable when the model is unavailable.

Methodologically, the project follows a layered analytics model. The early stages reduce raw data into a trustworthy form, the middle stages compute measurable patterns, and the later stages turn those patterns into human-readable findings. That mirrors the workflow of a human analyst while still keeping the process automated and repeatable.

The implementation approach is also incremental. Each agent was developed as a distinct module, which made it easier to test one stage at a time and identify integration issues early. This is a good fit for a mid-term report because it shows progress in the structure of the system, not only in the final output.

6. IMPLEMENTATION DETAILS

6.1. Agent 1: Structural Profiler

Purpose. Agent 1 (`agent1_structural_profiler`) reads the input file and records its structure. It deliberately performs no fixing and no inference; it only observes and records, giving later stages a factual baseline. The overall workflow is shown in Figure 6.1.

Inputs. It reads `csv_path` from the shared `GraphState`, an optional `sheet_name` for Excel inputs, and the running errors list.



Figure 6.1: Workflow of Agent 1: Structural Profiling

File loading. Loading is dispatched by extension in `_load_dataframe`. Excel files are read by `_read_excel_file`, which reads a pinned sheet directly or concatenates all sheets (outer join) while tagging each row with a `_source_sheet` column and warning on schema mismatch. CSV files are read by `_read_mixed_delimiter_csv`, which reads raw lines through `_read_csv_lines` using the encodings `utf-8-sig`, `cp1252`, and `latin-1` in turn.

That reader detects the delimiter per line by comparing comma and semicolon counts, rebuilds each row with Python’s `csv` module, and retries with the alternate delimiter when the field count does not match the header. Unknown extensions fall back to CSV parsing.

Structural profiling. Per column it records the Pandas dtype, missing count (`isna().sum()`), missing rate, unique count (`nunique(dropna=False)`), and the first three non-null `sample_values`. Dataset-level fields include shape, overall missing rate, and duplicate rows from `df.duplicated().sum()` with the duplicate rate.

Reliability. `_compute_agent1_confidence` produces a confidence value that decreases with missingness and duplication:

$$c_1 = \text{clamp}\left(1 - 0.5 \frac{m}{100} - 0.3 \frac{d}{100}, 0, 1\right) \quad (6.1)$$

where m is the overall missing rate and d the duplicate rate (percentages). The stage is marked ready when $c_1 \geq 0.8$ and `needs_review` otherwise, via `update_reliability`.

Output and interaction. On success it writes `raw_profile`, caches the `DataFrame` in `_df_cache`, and updates errors and reliability. If loading fails, the exception is appended to errors and the state returns early. The cached `DataFrame` and profile are consumed directly by Agent 2.

6.2. Agent 2: Semantic Tagger

Purpose. Agent 2 (agent2_semantic_tagger) produces the schema_blueprint, a per-column description of intended type, semantic role, and cleaning policy. When the LLM is invoked, the model is served locally through Ollama running a Llama-family Transformer. Figure 6.2 summarizes the flow.

Inputs. It uses raw_profile and the cached DataFrame _df_cache from Agent 1, plus the errors list. If no DataFrame is present, it records an error and returns.

```
[Agent 2] Type sniffing summary: {'unknown': 16, 'numeric': 5}
[Agent 2] Blueprint built for 21 columns
[Agent 2] Semantic tags by column:
- order_id: semantic_tag=identifier, intended_type=string, imputation=drop, encoding=None, identifier=True, analysis_allowed=True, confidence=70.0, null_action=drop_rows, suitable=True, reason=identifier_clean
- order_date: semantic_tag=datetime, intended_type=datetime, imputation=None, encoding=None, identifier=False, analysis_allowed=True, confidence=60.0, null_action=None, suitable=True, reason=datetime_missingness
- ship_date: semantic_tag=datetime, intended_type=datetime, imputation=None, encoding=None, identifier=False, analysis_allowed=True, confidence=60.0, null_action=None, suitable=True, reason=datetime_missingness
- delivery_date: semantic_tag=datetime, intended_type=datetime, imputation=None, encoding=None, identifier=False, analysis_allowed=True, confidence=60.0, null_action=None, suitable=True, reason=datetime_missingness
- order_status: semantic_tag=categorical_label, intended_type=string, imputation=mode, encoding=one_hot, identifier=False, analysis_allowed=True, confidence=60.0, null_action=impute_mode, suitable=True, reason=low_cardinality_category
- customer_id: semantic_tag=identifier, intended_type=string, imputation=drop, encoding=None, identifier=True, analysis_allowed=False, confidence=70.0, null_action=drop_rows, suitable=False, reason=identifier_duplicates
```

Figure 6.2: Workflow of Agent 2: Semantic Tagging

Type sniffing. Before any model call, _infer_intended_types performs cheap local inference, mapping each column to numeric, datetime, boolean, string, or unknown using the dtype and an 80% parseability threshold for object columns.

Prompt construction and inference. _build_llm_prompt builds a minimal prompt with each column's name, inferred type, missing rate, unique count, and three samples. A fixed system prompt requires the model to return *only* valid JSON describing intended_type, semantic_tag, is_identifier, scaling_allowed, imputation_strategy, encoding_strategy, and null_policy, at a low temperature (0.1) for near-deterministic output.

When the schema exceeds LLM_SINGLE_CALL_THRESHOLD (20) columns, _split_columns_into_batches splits it into batches of LLM_BATCH_SIZE (15). _call_llm_for_schema_blueprint_with_retry calls the model per batch and, if a truncated response fails JSON parsing, recursively halves the batch, retries, and merges results with _merge_schema_blueprints. _parse_schema_blueprint_response accepts raw or fenced JSON and, as a last resort, extracts the outermost { ... } span.

Enrichment and confidence. _apply_missingness_policy and _enrich_missingness_metadata fill any missing semantic_tag via _infer_semantic_tag_from_metadata and attach a column_assessment, null_policy, and encoding_strategy. Columns above MISSINGNESS_ANALYSIS_THRESHOLD_PCT (20%) are marked analysis_allowed = false. _calculate_semantic_confidence scores each tag from 0–100, and a __metadata__

block records the risk assessment and preprocessing recommendation.

Fallback and output. On invalid JSON or any exception, `_fallback_blueprint` assigns conservative types and policies from the sniffed types, keeping the pipeline running without the LLM. The agent writes `schema_blueprint` and errors; this blueprint drives every decision in Agent 3.

6.3. Agent 3: Preprocessor

Purpose. Agent 3 (`agent3_preprocessor`) performs deterministic cleaning and feature construction guided entirely by the `schema_blueprint`. It reads `_df_cache`, `schema_blueprint`, and `raw_profile`, returning early if either input is missing. The staged pipeline is shown in Figure 6.3.

Profile selection. `_build_preprocessing_config` selects a `strict`, `balanced`, or `lenient` profile. A pinned profile is honoured; otherwise `_detect_dataset_domain` inspects finance keywords and currency/percentage tags, choosing `strict` for finance/sales data. Each profile sets currency limits, a maximum tax rate, reconciliation tolerances, and quality weights.

```
[Agent 3] Preprocessing: profile=strict domain=finance_sales input=10000021
[Agent 3] Completed: rows=10000->10000 cols=21->115 quality=100.0/100 raw_missing=0.0% remaining_nulls=0.0%
[Agent 3] Cleaning summary: deduped=0, scaled=1, added cols=94, removed_rows=0
[Agent 3] Note: the cleaned dataset is normalized and exported to outputs/cleaned_data.csv; if the row count stays the same
one to drop.
[Agent 4] Starting analysis: 10000 rows x 115 cols
```

Figure 6.3: Workflow of Agent 3: Preprocessing Pipeline

Cleaning pipeline. Processing runs as an ordered, logged sequence. Step 0 removes exact duplicates with `dedup_exact_rows`, asserting the count matches Agent 1 and aborting via `_assert_row_survival_or_abort` if under half the rows survive. Step 1 (`_clean_currency_values`) uses `_normalize_currency_text` to convert parenthetical negatives, strip currency symbols and codes, and resolve European versus US decimals, writing a `_parse_failed` flag. Step 2 (`_coerce_types`) casts to intended types, tracking new NaNs; Step 3 standardizes text and Step 3b re-deduplicates after canonicalization.

Step 4 (`_impute`) handles missing values per each column's `null_policy/imputation_strategy`, supporting flag-only, identifier row drop, and mean, median, mode, or unknown-label fills. Step 5 encodes categoricals; Step 6 (`_clip_outliers`) applies `_adaptive_outlier_clipping` with percentile bounds for skewed/high-risk columns and the IQR $1.5\times$ rule otherwise; Step 7 (`_scale_columns`) applies Min–Max scaling while keeping `_raw` and `_scaled` copies in `scaling_params`. Step 8 (`_extract_date_features`) derives calendar features, and Step 9 (`_derive_business_metrics`) uses whole-word matching (`_find_col`) and an 80% numeric check to derive profit, margin, revenue per unit, total revenue, discount, budget variance, and total cost when possible.

Validation and quality. `_validate_count_ranges` and `_validate_financial_constraints` flag invalid counts and implausible tax rates into a `ColumnLedger`, and `_compute_enhanced_quality` combines remaining nulls, validation failures, and duplication into a 0–100 `data_quality` score.

Output and interaction. Every step appends to `preprocessing_log` with null diffs, and the cleaned frame is exported to `outputs/cleaned_data.csv`. The agent updates `cleaned_df`, `cleaned_csv_path`, `scaling_params`, `preprocessing_log`, `data_quality`, `column_ledger`, and `reliability`. Critical failures trigger `_early_exit_with_error`. The `cleaned_df` and `scaling_params` feed Agent 4.

6.4. Agent 4: Statistical Analysis

Purpose. Agent 4 (`agent4_analysis`) turns the cleaned dataset into descriptive statistics and chart artifacts. It reads `cleaned_df` and `schema_blueprint`, errors out if no cleaned frame exists, clears previous PNGs with `_clear_chart_dir`, and saves figures through `_save` into `outputs/charts`. Figure 6.4 outlines the analysis flow.

Column selection. `_numeric_cols` and `_categorical_cols` exclude validation columns (`_parse_failed`, `_range_failed`), scaling backups (`_raw`, `_scaled`), identifiers, datetime columns, and columns marked `analysis_allowed = false`, so analysis runs only on meaningful fields.

```
[Agent 4] Starting analysis: 10000 rows x 115 cols
[Agent 4] Step 1 – Descriptive stats: 27 columns
[Agent 4] Step 2 – Correlation done, strong pairs: 8
[Agent 4] Step 3 – Growth rates skipped (no revenue/time axis)
[Agent 4] Step 4 – Rankings skipped (no revenue/category pairing)
[Agent 4] Step 5 – Seasonality skipped (no time axis)
[Agent 4] Step 6 – Anomalies: 235 unique rows (2.35%) across 1 column
[Agent 4] Step 7 – Distributions done (4 charts)
[Agent 4] Step 8 – Regression done (27 columns, 0 charts)
[Agent 4] Step 9 – Distribution charts done (1 charts)
[Agent 4] Step 10 – Derived metrics skipped (no derived metrics)
[Agent 4] Done – 6 charts saved to outputs/charts/
```

Figure 6.4: Workflow of Agent 4: Statistical Analysis

Analyses. `_descriptive_stats` computes count, mean, median, standard deviation, variance, extremes, quartiles, skewness, and kurtosis. `_correlation` computes Pearson and Spearman matrices, lists strong pairs with $|r| \geq 0.5$ (strong when ≥ 0.7), and renders a heatmap. `_growth_rates` uses a revenue column with `_month/_year` features for month- and quarter-over-quarter charts, `_top_bottom_rankings` ranks revenue across up to three categories, and `_seasonality` plots monthly averages with best/worst months.

`_detect_anomalies` flags values whose absolute z -score exceeds 3.5:

$$z = \frac{x - \mu}{\sigma} \quad (6.2)$$

where x is a value and μ, σ the column mean and standard deviation. `_category_distributions` charts categories with at most 15 levels, and `_regression_trends` fits an OLS line on the `_month` axis with `scipy.stats.linregress`, recording slope, intercept, R^2 , p -value, and significance ($p < 0.05$). `_distribution_charts` draws a box plot and revenue histogram, and `_derived_metrics_charts` plots profit margin over time and a derived-metrics summary.

Output and interaction. The agent writes the stats dictionary and `chart_paths`, updates errors, and records reliability (0.9 when statistics were produced, otherwise 0.4). These statistics and charts form the evidence base consumed by the validation and reporting stages.

6.5. Current Implementation Snapshot

Table 6.1: Implementation snapshot at the mid-point

Module	Current state	Remarks
Graph state and routing	Complete	Conditional pipeline is in place
Agent 1 to Agent 3	Complete	Profiling, tagging, and pre-processing are working
Agent 4	Complete	Charts and statistics are generated
Agent 5	In progress	Validation gate not yet implemented
Agent 6	In progress	HTML report generation not yet implemented
UI and deployment polish	In progress	Still needs refinement
Test expansion	In progress	More edge cases are being added

7. RESULTS AND ANALYSIS

7.1. Observed Progress

The current codebase already demonstrates an end-to-end pipeline from raw data ingestion through statistical analysis and chart generation. In the sample execution log, Agent 4 generates multiple chart types after profiling, semantic tagging, and preprocessing complete. That means the system is no longer just a design proposal; it is now a working backend prototype up to Agent 4.

That progress is important because it confirms the proposal is executable rather than purely conceptual. The pipeline is already performing the core work the proposal promised: profiling, semantic tagging, preprocessing, and statistical analysis. The remaining validation and reporting stages are still to be built, which is appropriate for a mid-term checkpoint.

7.2. Practical Output

The most visible outputs produced by the current implementation are:

- a structural profile of the uploaded dataset,
- a semantic schema blueprint,
- a cleaned dataset with quality metrics and an audit trail,
- statistical summaries and charts,
- per-stage reliability metadata with confidence scores.

These outputs are useful because they show both machine-readable state and human-readable artifacts. The system can therefore be inspected by later agents as well as by project evaluators. That dual nature is one of the most practical strengths of the current implementation.

7.3. Analysis

The strongest result so far is that the system separates concerns well. Agent 1 does not try to clean data, Agent 3 does not try to infer semantics, and Agent 4 focuses only on analysis rather than cleaning. That separation makes the pipeline easier to maintain and easier to explain in a report. Another important result is the reliability mechanism: the system keeps track of confidence and decision readiness instead of assuming every stage succeeds equally well.

From an engineering perspective, the current codebase is already behaving like a real analytics pipeline. It does not just transform rows and columns; it builds a story about the dataset in stages. It starts with raw structure, adds meaning, cleans the data, and computes evidence in the form of statistics and charts. The later stages that check that evidence and produce a report are planned as the next steps. That staged sequence is the core technical value of the project.

The depth of the implementation is also visible in the way the backend handles data quality. Rather than relying on one generic cleaning rule, the pipeline uses multiple checks and logs to capture what changed and why. That makes the output more defensible and more appropriate for business datasets where context matters.

7.4. Current Challenges

The main remaining work spans both new functionality and refinement. The two unimplemented agents (validation and reporting) still need to be built, and the pipeline also needs broader testing on different dataset shapes and a clearer evaluation section for the final report. The planned validation gate will need careful tuning so that it remains strict without rejecting acceptable outputs unnecessarily.

Another challenge is balancing flexibility and consistency. The system needs to handle varied datasets, but it also needs to produce stable and repeatable results. That balance is important because the final project should be practical without becoming unpredictable.

At this stage, the project is best described as functional but still maturing. The core behavior is already present, but the last phase needs stronger evaluation, documentation, and presentation work before submission.

8. REMAINING TASKS

1. Implement Agent 5 (Validation and Quality Guardrail) to check the statistical outputs, charts, and quality metrics and to gate low-confidence results before reporting.
2. Implement Agent 6 (Report Generation) to assemble the validated statistics and charts into a self-contained final report.
3. Improve the presentation layer and decide whether a user-facing interface should be included in the final deliverable.
4. Expand automated tests across preprocessing and the new validation and reporting stages.
5. Prepare the final evaluation section with sample outputs and documented observations, then clean up the report language, citations, and formatting before final submission.

In addition to these tasks, the team needs to keep the report aligned with the actual backend behavior. As the codebase evolves, the document should continue to describe what the system really does rather than only what was originally planned. That alignment will matter in the final defense because it keeps the report credible and technically consistent.

9. CONCLUSION

The project has reached a strong mid-point. The core backend pipeline is implemented through Agent 4, the agents are coordinated through shared state, and the system can process data through profiling, semantic tagging, preprocessing, and statistical analysis. Validation (Agent 5) and report generation (Agent 6) remain to be implemented, so the remaining work combines this new functionality with refinement of the existing stages. This gives the project a solid base for final evaluation and presentation.

At this stage, the project can already be described as a functioning multi-agent analytics backend up to the statistical analysis stage. Its main value lies in combining deterministic processing with controlled language-model assistance while keeping the workflow auditable. The next phase is to build the validation and reporting agents and to strengthen the presentation, test coverage, and evaluation so that the final report can present the system confidently and with evidence.

A. COST ESTIMATE

Table A.1: Indicative project cost estimate

Item	Estimate	Notes
Development laptop and local machine	Existing	Already available for development
Python, Pandas, LangGraph, plotting stack	Existing	Open-source software stack
LLM API usage	Variable	Depends on testing and report generation
Internet and miscellaneous expenses	Variable	Project maintenance cost

The cost estimate is intentionally lightweight because the project uses open-source libraries and an existing development environment. The main variable cost comes from API usage during testing and report generation. In practice, that cost can be managed by keeping the heuristic fallback path available and by avoiding unnecessary repeated model calls.

B. PROJECT TIMELINE

Table B.1: Simplified project timeline

Task	Status	Phase
Requirement analysis and proposal	Complete	Initial
Agent implementation and integration	Mostly complete	Development
Validation and reporting refinement	In progress	Development
Testing and evaluation	In progress	Finalization
Report submission preparation	Pending	Final

This timeline reflects the actual progression of the work. The first phase focused on design and planning, the current phase focuses on integration and documentation, and the final phase will focus on evaluation and submission readiness. It provides a concise view of how the project has moved from proposal to implementation.

REFERENCES

- [1] Y. K. Dwivedi, L. Hughes, A. M. Baabdullah, et al., “So what if ChatGPT wrote it? Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy,” *International Journal of Information Management*, vol. 71, 2023.
- [2] OpenAI, “GPT-4 Technical Report,” 2024.
- [3] M. Sun, R. Han, B. Jiang, H. Qi, D. Sun, Y. Yuan, and J. Huang, “LAMBDA: A Large Model Based Data Agent,” arXiv preprint arXiv:2407.17535, 2024.
- [4] Y. K. Liu, et al., “Summary of ChatGPT-Related Research and Perspective Towards the Future of Large Language Models,” arXiv:2306.07209, 2023.
- [5] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On Faithfulness and Factuality in Abstractive Summarization,” in *Proceedings of ACL*, 2020.
- [6] V. Dibia and C. Demiralp, “Data2Vis: Automatic Generation of Data Visualizations Using Sequence-to-Sequence Recurrent Neural Networks,” *IEEE Computer Graphics and Applications*, vol. 39, no. 5, pp. 33–46, 2019.
- [7] K. Hu, M. A. Bakker, S. Li, T. Kraska, and C. A. Hidalgo, “VizML: A Machine Learning Approach to Visualization Recommendation,” in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2019.
- [8] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer, “Formalizing Visualization Design Knowledge as Constraints: Actionable and Extensible Models in Draco,” *IEEE Transactions on Visualization and Computer Graphics*, 2018.
- [9] Y. Luo, X. Qin, N. Tang, and G. Li, “DeepEye: Towards Automatic Data Visualization,” in *IEEE International Conference on Data Engineering (ICDE)*, 2018.
- [10] LangChain, “LangGraph: A Python Library for Building Stateful, Multi-Agent Applications,” 2024.
- [11] Q. Wu, G. Bansal, J. Zhang, et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework,” 2023.
- [12] Z. Ji, et al., “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, 2023.

- [13] G. Li, et al., “CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society,” arXiv:2303.17760, 2023.
- [14] N. Shinn, et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” arXiv:2303.11366, 2023.
- [15] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *ICLR*, 2015.
- [16] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.